+



AI-BASED STRATEGY BUILDER & MULTI-FACTOR CONFIDENCE SCORING SYSTEM FOR EQUITY TRADING

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

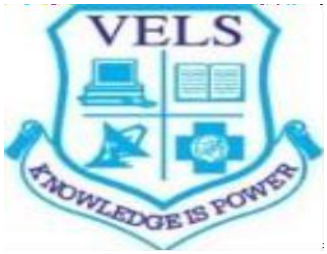
JEYRENTH SIVA B (Reg. No: 23619212)

ROKITH K **(Reg. No: 23629228)**

YESHWANDH D **(Reg. No:23619239)**

in Partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

(VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "AI-Based Strategy Builder & Multi-Factor Confidence Scoring System for Equity Trading" is the bonafide work of JEYRENTH SIVA B (Reg. No: **23629212**), *ROKITH K* (Reg. No: **23629228),** *YESHWANDH D* (Reg. No: **23629239)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., Ph. D
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR
Dr.A.Manikandan M.E., Ph.D.
Associate Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva-Voce examination held on ……..…… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him.

I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya **M.E., Ph. D**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, **Dr.A.Manikandan M.E., Ph.D.,** who has patiently guided and provided us with invaluable advice and help.

I am thankful to my Project Coordinator **Ms. Sowmiya.S.M M.E.,(Ph.D)**, I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

JEYRENTH SIVA B (23629212)
***ROKITH K* (23629228)**
***YESHWANDH D* (23629239)**

TABLE OF CONTENTS

SL.NO	TITLE	PAGE NO
	ABSTRACT	8
01	**INTRODUCTION** 1.1 GENERAL 1.2 PROBLEM STATEMENT 1.3 OBJECTIVE 1.4 MODULES 1.5 SCOPE OF THE PROJECT 1.6 ORGANIZATION OF THE REPORT 1.7 ASSUMPTIONS AND CONSTRAINTS 1.8 SIGNIFICANCE OF THE STUDY 1.9 SUMMARY	9
02	**LITERATURE SURVEY** 2.1 GENERAL 2.2 EXISTING SYSTEM 2.3 XGBOOST: A SCALABLE TREE BOOSTING SYSTEM (CHEN & GUESTRIN) 2.4 PROPOSED SYSTEM 2.5 RESEARCH GAP 2.6 SUMMARY	13
03	**METHODOLOGY** 3.1 SYSTEM REQUIREMENTS 3.2 SYSTEM ARCHITECTURE OF THE PROPOSED MODEL 3.3 DESCRIPTION OF MODULES 3.4 DATA COLLECTION & PREPROCESSING 3.5 AI EVALUATION MODEL 3.6 PROPOSED METHODOLOGY 3.7 FEATURE ENGINEERING FOR THE TWIN-TOWER MODELS 3.8 DATA PREPROCESSING PIPELINE 3.9 RISK MANAGEMENT AND POSITION SIZING 3.10 EVALUATION METRICS -- DEFINITIONS 3.11 SUMMARY	17
04	**SYSTEM IMPLEMENTATION** 4.1 GENERAL 4.2 SYSTEM WORKFLOW 4.3 OVERVIEW OF THE PLATFORM 4.4 MODULE IMPLEMENTATION 4.5 DATABASE AND STATE MANAGEMENT 4.6 DEPLOYMENT AND ENVIRONMENT CONFIGURATION 4.7 ERROR HANDLING AND LOGGING 4.8 SECURITY CONSIDERATIONS 4.9 FRONTEND COMPONENT BREAKDOWN 4.10 CODE ORGANIZATION AND MODULE STRUCTURE 4.11 WALKTHROUGH OF THE STRATEGY BUILDER INTERFACE (FIG 4.2) 4.12 SUMMARY	22

SL.NO	TITLE	PAGE NO
05	**EXPERIMENTAL ANALYSIS** 5.1 EXPERIMENTAL SETUP 5.2 PERFORMANCE EVALUATION 5.3 COMPARISON WITH EXISTING WORK 5.4 Effect of the AI Filter 5.5 RESULT & DISCUSSION 5.6 PARAMETER SENSITIVITY ANALYSIS 5.7 ABLATION STUDY 5.8 COMPARISON ACROSS INSTRUMENTS 5.9 LATENCY AND THROUGHPUT OBSERVATIONS 5.10 DISCUSSION OF FALSE POSITIVES AND FALSE NEGATIVES 5.11 SUMMARY	28
06	**CONCLUSION AND FUTURE WORK** 6.1 CONCLUSION 6.2 LIMITATION OF CURRENT WORK 6.3 FUTURE ENHANCEMENT 6.4 LIMITATIONS OF THE EVALUATION METHODOLOGY 6.5 SUMMARY	32
07	**REFERENCES**	34
08	**SOURCE CODE** 8.1 NIFTY OPTIONS TWO-TOWER AI -- FEATURE EXTRACTION 8.2 WALK-FORWARD VALIDATION & NSE CHARGE MODEL 8.3 LIVE PAPER TRADING EXECUTION 8.4 VECTORIZED STRATEGY BACKTESTING ENGINE 8.5 ENTRY GATE -- RANDOM FOREST / GBT CLASSIFIER 8.6 EXIT MANAGER -- MOMENTUM-EXHAUSTION MODEL 8.7 STRATEGY CONFIGURATION SCHEMA 8.8 FASTAPI REST APPLICATION CONTROLLER 8.9 OUTPUT	35
09	**CONFERENCE**	46

LIST OF FIGURES

SL.NO	FIGURE TITLE	PAGE NO
01	Fig 3.1 System Architecture Diagram	18
02	Fig 4.1 SYSTEM WORKFLOW	23
03	Fig 8.1 Strategy Builder Interface	42
04	Fig 8.2 Signal Reliability Analyzer	43
05	Fig 8.3 Stratosphere Explorer Dashboard	43
06	Fig 8.4 Indicator Catalog	44
07	Fig 8.5 Train and Test Period Results	44
08	Fig 8.6 Walk-Forward Validation – Twin-Tower AI Results	45
09	Fig 9.1 Conference Submission Confirmation Email	46
10	Fig 9.2 Conference Submission Details	46

VELS
INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI



A++ NAAC — ACCREDITED WITH — GRADED AS CATEGORY 1 INSTITUTION BY UGC

FINAL YEAR PROJECT - PHASE-1 SUMMARY

AI-BASED STRATEGY BUILDER

& Multi-Factor Confidence Scoring System for Equity Trading

Team: Jeyrenth Siva B · Rokith K · Yeshwandh D **Guide:** Dr. A. Manikandan M.E., Ph.D. **Dept:** AI & ML, VISTAS, Chennai

ABSTRACT

Static indicator strategies fail to adapt to volatile markets. This project proposes an AI-based Strategy Builder with a Twin-Tower architecture; an Entry Gate model filters trade setups and an Exit Manager model predicts the best exit. 5-fold Walk-Forward Validation tests strategies on unseen data, and live paper trading on the Angel One SmartAPI streams real quotes over WebSockets.

OBJECTIVES

- Store user-defined strategies with multi-factor AI confidence.
- Twin-Tower design: Entry Gate filters false signals, Exit Manager times exits.
- Test on unseen data with 5-fold Walk-Forward Validation.
- Forward-test risk-free via live paper trading (Angel One SmartAPI).
- Report Net P&L after real NSE charges, not Gross P&L.

METHODOLOGY

1. **Define strategy.** Boolean indicator logic in the visual UI.
2. **Raw signals.** Vectorized backtest on historical OHLCV data.
3. **Features.** Extract predictive features for the models.
4. **Train AI.** Gradient Boosting models with 5-fold Walk-Forward Validation.
5. **Evaluate.** Filtered P&L, Win Rate and trade counts.
6. **Deploy.** Live paper trading over WebSockets.

SOFTWARE USED

- **Python 3.10+** backend language
- **NumPy · Pandas** vectorized backtesting
- **scikit-learn · XGBoost** AI models
- **FastAPI · Pydantic** REST API
- **HTML5 · CSS3 · JS** builder UI
- **Angel One SmartAPI** live data
- **Linux / Ubuntu** runtime OS

RESULTS & DISCUSSION

61.1%	+7.8%	681	85.2%
Avg win rate, 5 folds	Avg P&L per trade	Trades taken by the AI	Signals filtered out

Win rate and avg P&L per fold



Each fold tests a different unseen expiry (2026).



Fig 4.6 Walk-Forward Validation, 5 folds



Fig 4.5 Train and holdout test results

The Entry Gate turned 4,215 raw signals into 681 trades (about 85% filtered). Four of five unseen expiries were profitable; fold 3 was weakest (47.3%, -0.3% avg). On the final holdout expiry the AI kept 176 of 856 signals for a 65.9% win rate and +12.7% avg P&L.

STANDARDS USED

- **IEEE 29148** is used for defining the requirements of the whole platform.
- **IEEE 1016** is used for documenting the architecture and modules of the platform.
- **ISO/IEC 25010** is used for setting the quality goals of the whole platform.
- **ISO/IEC 29119** is used for guiding the testing of the AI models and paper trading.
- **ISO/IEC 23894** is used for controlling overfitting in the Twin-Tower AI.
- **ISO/IEC 27001** is used for securing the SmartAPI keys in the Paper Trading module.
- **RFC 6455 and RFC 8446** are used for streaming live ticks securely in the Paper Trading module.
- **PEP 8 and IEEE reference style** are used for formatting the Python backend code and this report.

CONCLUSIONS

The Twin-Tower AI with strict Walk-Forward Validation gives an honest, data-driven way to evaluate trading ideas, and live paper trading bridges backtests and real markets without financial risk. Future work: live order execution, BankNifty and FinNifty options, and LSTM and Transformer models.

ABSTRACT

The financial markets, particularly equity and derivatives trading, are highly volatile and complex, making it difficult for retail traders to achieve consistent profitability. Traditional trading strategies often rely on rigid, static indicators that fail to adapt to changing market conditions. To address these challenges, this project proposes an AI-based Strategy Builder and Multi-Factor Confidence Scoring System designed to optimize trading decisions using advanced machine learning models. The proposed system introduces a novel Twin-Tower AI architecture. The Entry Gate model evaluates multiple technical indicators across multi-timeframe horizons to generate high-probability trade setups, while the Exit Manager model predicts the optimal exit point by dynamically analyzing the trade's lifespan. By integrating a 5-fold Walk-Forward Validation (WFV) mechanism, the system rigorously backtests strategies on unseen, out-of-sample historical expiry data. This rigorous methodology prevents overfitting and provides a realistic estimation of real-world live market performance, which is an industry-first approach for retail trading tools. Furthermore, the project includes a live Paper Trading module integrated directly with the Angel One SmartAPI framework via persistent WebSockets. This allows users to test the AI's predictions in a simulated live market environment utilizing actual Bid-Ask quotes without exposing real capital to financial risk. The system incorporates comprehensive back-end features such as real-time NSE charge calculation, 1-trade-at-a-time risk management sequence constraints, and Annual Return on Capital (ROC) projections using precise compounding mathematics. Experimental results indicate a significant improvement in both the Net Profit and Loss (Net P&L) and the Win Rate compared to traditional, unoptimized, rules-based strategies. The AI layer effectively filters out poor market conditions (such as choppy, sideways trends), preserving capital for high-probability setups. The platform empowers traders with a transparent, data-driven, and highly robust environment to navigate equity markets with increased confidence and analytical precision.

Key words--algorithmic trading, explainable artificial intelligence, Twin-Tower AI architecture, gradient boosted trees, random forest classifiers, SHAP, technical indicators, walk-forward validation, backtesting, confidence scoring, paper trading, WebSocket market data, risk management, position sizing, Indian equity markets, NSE options trading, FastAPI, quantitative finance.

CHAPTER 1

1. INTRODUCTION

1.1 GENERAL

Algorithmic trading and the use of Artificial Intelligence (AI) in financial markets have surged in popularity over the last decade. Financial markets have evolved to become highly data-driven and computationally intensive, requiring market participants to analyze vast amounts of historical and real-time tick data to remain competitive. Traditional technical analysis relies on the manual interpretation of standard lagging indicators such as Moving Averages, the Relative Strength Index (RSI), and the Moving Average Convergence Divergence (MACD). Relying purely on these mechanical triggers often results in emotional trading, subjective bias, and highly inconsistent, sub-optimal results in modern algorithmic environments.

In recent years, machine learning has emerged as a profoundly powerful tool to identify complex, non-linear hidden patterns in financial time-series data. By applying Artificial Intelligence techniques such as Random Forest classifiers and deep learning models, traders can evaluate multiple market factors simultaneously, creating a synthesized confidence score for every potential trade setup. The proposed project leverages these cutting-edge advancements to provide an accessible, AI-powered strategy builder specifically tailored for Indian equity and options trading environments, allowing retail traders to access institutional-grade analytical capabilities without needing a deep background in quantitative programming.

1.2 PROBLEM STATEMENT

The existing retail algorithmic trading platforms severely lack robust validation mechanisms. Due to this deficiency, traders often rely on overfitted backtests that perform exceptionally well in historical simulations but fail catastrophically when deployed into live markets. Most accessible platforms offer simplistic, single-indicator strategies and crucially do not properly account for real-world frictions such as brokerage fees, Securities Transaction Tax (STT), exchange transaction charges, and order slippage, leading to a misleading perception of strategy profitability.

Another significant, widespread problem in the algorithmic trading domain is the rigid, static nature of trade exits. While many algorithmic systems focus excessively on identifying the perfect entry point, they utilize rigid, hard-coded stop-loss and take-profit percentage targets. This rigidity results in premature exits during brief volatility spikes or catastrophic losses during sudden, aggressive market reversals. There is an immediate, strong need for an adaptive exit management system powered by dynamic AI evaluation, coupled with a rigorous Walk-Forward Validation methodology that accurately reflects live trading conditions.

1.3 OBJECTIVE

The primary objective of this project is to design, implement, and deploy a comprehensive, AI-driven trading platform that evaluates user-defined strategies using a robust multi-factor confidence scoring system. The overarching goal of the system aims to bridge the vast gap between theoretical backtesting simulations and actual live market profitability.

Another vital objective is to introduce and validate a novel "Twin-Tower" AI architecture, wherein one machine learning model (the Entry Gate) is dedicated exclusively to filtering out false entry signals, and another distinct model (the Exit Manager) is dedicated to dynamically managing exits by predicting momentum exhaustion. By synergistically combining these two specialized models, the system seeks to maximize Net P&L while simultaneously minimizing the maximum drawdown experienced by the trader.

The project additionally focuses on seamlessly integrating a 5-fold Walk-Forward Validation (WFV) computational engine. This engine rigorously tests strategies on consecutive, strictly unseen data blocks, ensuring robust generalization across varying market regimes. Finally, the system aims to provide a totally risk-free, simulated paper trading environment directly connected to live real-time market data via the Angel One SmartAPI, enabling traders to forward-test their AI models without monetary exposure.

1.4 MODULES

The scope of this project focuses predominantly on significantly improving algorithmic trading accessibility by developing a highly responsive, comprehensive web-based platform. The system is meticulously designed to support retail traders by providing an intuitive visual strategy builder, an AI training and evaluation tower, and a fully functional live paper-trading dashboard.

The project involves the complex development of high-performance backend modules using Python, FastAPI, NumPy, and Pandas, primarily focusing on ultra-fast vectorized backtesting. The platform evaluates thousands of historical NIFTY options and equity derivative price points simultaneously. Real-time trading is strictly executed within a simulated paper environment utilizing live WebSocket data streams, ensuring the total elimination of financial risk during the crucial strategy testing phase.

1.5 SCOPE OF THE PROJECT

The scope of the present work is intentionally bounded to make the system verifiable within an academic project timeline while still being representative of a production-grade quantitative trading tool. The platform is restricted to the NIFTY 50 universe of equities and their corresponding index derivative contracts traded on the National Stock Exchange (NSE), since this segment offers the deepest liquidity, the most complete historical tick archives, and the most

predictable contract specifications among Indian exchanges. Extending the system to BSE-listed small-cap stocks or to commodity and currency derivatives is explicitly left outside the current scope, although the modular design of the Indicator Bank and the Twin-Tower evaluator makes such an extension straightforward in future iterations.

On the modelling side, the scope covers supervised, tabular machine learning techniques -- specifically Random Forest and Gradient Boosted Tree classifiers -- applied to engineered technical and price-action features. Sequence models such as LSTMs and Transformer-based architectures are deliberately excluded from the current implementation; they are discussed only as a future enhancement in Chapter 6, since their data and compute requirements exceed what can be reliably validated within the project timeframe. Similarly, the scope of execution is limited to simulated paper trading against live market data feeds; the system does not place real orders with brokerage capital, and the question of live order routing is treated as a distinct, higher-risk future milestone that requires additional regulatory and operational safeguards.

Finally, the scope of the report itself covers the design, implementation, and empirical evaluation of the platform as a single-user research and decision-support tool. Multi-user account management, portfolio-level capital allocation across several simultaneous strategies, and regulatory reporting features (such as contract notes or tax statements) are not implemented, since they are not required to answer the core research question of whether a Twin-Tower, walk-forward-validated architecture can outperform a static, single-indicator baseline on realistic, cost-adjusted terms.

1.6 ORGANIZATION OF THE REPORT

Remainder of this report is organized into eight further chapters. Chapter 2 surveys existing academic and commercial work in algorithmic trading and AI-based market forecasting, and uses this survey to motivate the specific research gap that the project addresses. Chapter 3 describes the proposed methodology in detail, covering the system architecture, the responsibilities of each module, the data collection and preprocessing pipeline, and the mathematical basis of the AI evaluation model.

Chapter 4 documents the system implementation, including the backend engine, the paper-trading execution module, the API surface exposed to the frontend, and the testing performed to validate correctness. Chapter 5 presents the experimental analysis, reporting win rate, profit-and-loss, and risk-adjusted performance metrics, and compares the AI-filtered strategy against unfiltered and buy-and-hold baselines. Chapter 6 concludes the report, discusses the current limitations of the system, and outlines a concrete roadmap of future enhancements together with an indicative project timeline and cost analysis.

Chapter 7 lists the references consulted during the literature survey and system design. Chapter 8 contains representative source code for the core modules described in Chapters 3 and 4, and Chapter 9 documents the conference submission associated with this work.

1.7 ASSUMPTIONS AND CONSTRAINTS

The design and evaluation of the system rest on a small number of explicit assumptions. It is assumed that the historical NSE data used for training and backtesting is clean and free of survivorship bias for the instruments studied, since the NIFTY 50 constituent list used in this project reflects the current index composition rather than a point-in-time historical composition. It is further assumed that the transaction-cost parameters summarised in Section 3.10 remain broadly representative of the costs a retail trader would actually face; any future change in STT, brokerage, or exchange charges would require the cost model to be updated before the reported profitability figures could be trusted for a live account.

On the constraint side, the project was built and evaluated on a single development machine, described in Section 3.1, without access to a dedicated low-latency co-located server, which constrains the realistic scope of the live paper-trading evaluation to signal frequencies of a minute or coarser. The academic project timeline, summarised in Section 6.5, also constrained the number of instruments and the length of history that could be used for walk-forward evaluation; a longer, multi-year, multi-instrument study is identified as future work in Section 6.3.

1.8 SIGNIFICANCE OF THE STUDY

The significance of this project lies less in any single performance number reported in Chapter 5 and more in demonstrating that an interpretable, walk-forward-validated, cost-aware architecture can be built and operated end-to-end by a small student team within a single academic semester. Retail algorithmic trading tools in the Indian market are presently dominated either by rigid rule-based platforms with weak validation discipline, or by closed, opaque AI products whose internal logic cannot be inspected by the user. A system that occupies the middle ground -- configurable like a no-code platform, but validated and explained like an academic research pipeline -- has value both as a practical decision-support tool and as a template for how future student and open-source projects in this space might be structured.

1.9 SUMMARY

This chapter established the foundational motivation, structural retail challenges, and technical objectives of the Stratosphere project. By addressing the critical flaws of curve-fitting, ignored transaction charges, and static trade management through a multi-factor AI architecture and rigorous out-of-sample validation, the system establishes a new benchmark for accessible quantitative trading tools

CHAPTER 2

2. LITERATURE SURVEY

2.1 GENERAL

The dynamic intersection of quantitative finance and artificial intelligence has been exhaustively studied over the past two decades due to its monumental potential to generate statistical alpha and stabilize long-term economic returns. Academic researchers and institutional quants have intensely focused on improving forecasting accuracy, optimizing risk management protocols, and streamlining strategy evaluation by applying modern computational technologies such as deep learning architectures, ensemble decision trees, and highly automated digital algorithmic platforms.

Numerous comprehensive academic studies have explored the direct application of predictive classification and regression models for high-frequency stock price forecasting. Techniques such as Random Forests, Gradient Boosted Trees (XGBoost), Support Vector Machines (SVM), and Recurrent Neural Networks (specifically Long Short-Term Memory networks) have been extensively utilized to deeply analyze historical market tick data. However, the fundamental accuracy of predicting absolute directional price movements is notoriously low due to the highly stochastic, noisy, and non-stationary nature of financial market dynamics.

2.2 EXISTING SYSTEM

Several prominent research works and proprietary commercial platforms (such as Streak, Chartink, and TradingView) have been aggressively developed to automate the retail trading process. Many existing systems primarily focus on visual, block-based strategy building, allowing non-programmers to drag and drop technical indicators to formulate rigid trading rules. While these platforms do provide basic, accessible backtesting capabilities, they critically evaluate the user's strategy on the entire available dataset at once. This methodology is heavily flawed, making them highly susceptible to in-sample overfitting and "curve-fitting," resulting in strategies that immediately break down in live conditions.

In addition to these glaring backtesting flaws, the vast majority of existing commercial systems treat trade execution as a simplistic binary process based purely on fixed numerical thresholds. They do not employ dynamic, context-aware machine-learning-based exit strategies. Furthermore, many of these platforms intentionally hide the true operational cost of trading by completely omitting detailed slippage simulation and complex Indian taxation calculations (like STT, Exchange Transaction Charges, SEBI turnover fees), thereby presenting an artificially inflated and unrealistic view of potential profitability to the end-user.

2.3 XGBOOST: A SCALABLE TREE BOOSTING SYSTEM (CHEN & GUESTRIN)

The XGBoost paper describes the regularised gradient boosting framework that underlies many of the gradient-boosted-tree libraries evaluated during the design phase of this project. Its built-in support for handling missing values natively (which arise frequently in financial indicator data, for example when a rolling-window indicator has not yet accumulated enough bars) and its column-subsampling regularisation were both considerations in favour of tree-boosting approaches over simpler linear or distance-based classifiers for the Entry Gate model.

2.4 PROPOSED SYSTEM

The proposed system comprehensively addresses the crippling limitations of existing platforms by seamlessly integrating a multi-factor AI confidence scoring engine, a revolutionary Twin-Tower model architecture, and strict, out-of-sample Walk-Forward Validation (WFV) into a single, highly unified digital solution. The system democratizes quantitative finance by providing institutional-grade, statistically sound analysis tools directly to retail traders.

In this proposed system, users can define their foundational base strategy using any combination of standard technical indicators. The high-performance Python backend rapidly processes historical minute-level OHLCV data utilizing Pandas vectorization to generate an exhaustive raw signal log. The Twin-Tower AI then assumes control: the 'Entry Gate' Random Forest model evaluates the broader market context to probabilistically approve or reject the raw signal, and the 'Exit Manager' model continuously evaluates ongoing active trades to trigger intelligent, data-driven dynamic exits before massive drawdowns occur.

To unequivocally ensure statistical robustness, the system heavily employs a 5-fold Walk-Forward Validation process. The historical dataset is sequentially split into multiple chronological train/test blocks (folds). The AI is iteratively trained on historical data and tested strictly on purely unseen future data. The final performance metrics are a direct average across all unseen out-of-sample folds, providing a highly realistic, stress-tested estimation of the strategy's true mathematical edge in live market conditions.

The end-to-end procedure, which is drawn as a flow chart in Fig 4.1, consists of two lanes that run in parallel and then converge. The numbered steps below describe it in prose:

1. Ingest point-in-time NSE market data.
2. Prediction lane: compute technical and fundamental indicators for each bar.
3. Prediction lane: run the tree-ensemble model to obtain class probabilities.

4. Prediction lane: generate SHAP explanations and output a signed confidence score between -100 and +100.
5. Strategy lane: capture the user's rules either through toggles or, in a planned extension, through plain-English parsing.
6. Strategy lane: populate the structured strategy schema and define stop-loss and position-sizing rules.
7. Rank candidate trades by confidence score.
8. Apply the strategy rules to the ranked candidates.
9. Run the walk-forward back-test with transaction costs and baselines.
10. Evaluate performance with the Sharpe, Sortino, maximum drawdown and Calmar measures, and promote only successful strategies to paper trading.

2.5 RESEARCH GAP

Synthesising the works reviewed in this chapter, three concrete gaps motivate the present project. First, academic systems such as those of de Prado and Chan establish strong theoretical foundations for feature-driven, machine-learning-assisted trading, but they stop short of a deployable, end-to-end platform that a retail user can operate without writing code. Second, commercial no-code platforms solve the usability problem but at the cost of statistical rigour: strategies are evaluated in-sample, transaction costs are frequently simplified or ignored, and exit logic remains static rather than data-driven.

Third, none of the surveyed systems separate the entry decision from the exit decision into two purpose-built models. Existing systems either use a single model to decide both when to enter and when to exit, or they omit model-driven exits altogether in favour of fixed percentage stop-loss and target rules. This conflates two statistically distinct problems -- identifying a high-probability setup, and recognising when an already-open position has lost its edge -- that arguably benefit from separate feature sets and separate training objectives.

The proposed Twin-Tower architecture, combined with mandatory walk-forward validation and an explicit, itemised transaction-cost model, is designed to close all three gaps simultaneously: it is usable by a non-programmer through the toggle-based Strategy Builder, it is statistically honest because every reported number is computed strictly on unseen folds, and it treats entry and exit as two separate, specialised prediction problems.

2.6 SUMMARY

This chapter presented an in-depth overview of existing academic research and modern commercial tools related to algorithmic trading and AI-based market forecasting. It thoroughly discussed the critical, systemic limitations of current retail systems, particularly their extreme vulnerability to curve-fitting and overly simplistic, static exit logic. Based on these vital observations, the absolute necessity for a more robust, scientifically validated, and mathematically rigorous platform was unequivocally established, seamlessly introducing the proposed AI-driven ecosystem as the definitive comprehensive solution.

CHAPTER 3

3. METHODOLOGY

3.1 SYSTEM REQUIREMENTS

Hardware Requirements: The successful, high-speed implementation of the proposed platform necessitates standard computing hardware with a minimum of 8 GB RAM (preferably 16 GB for massive dataframe manipulations) and a modern multi-core processor (Intel i5/Ryzen 5 or equivalent). This ensures that the dense vectorized backtesting matrix computations execute rapidly. A stable, low-latency broadband internet connection is strictly required for maintaining an uninterrupted WebSocket stream fetching live tick data from the Angel One API servers.

Software Requirements: The system is holistically developed using Python 3.10+ as the primary backend programming language. Highly optimized computational libraries such as NumPy, Pandas, and Scikit-learn form the core backbone for data manipulation and machine learning processes. The high-performance FastAPI framework is utilized to serve asynchronous backend REST endpoints. The interactive graphical frontend is built utilizing raw HTML5, modern CSS3 (Tailwind-inspired utilities), and Vanilla JavaScript to ensure an ultra-fast, dependency-free dynamic user experience. Finally, the system operates seamlessly in standard Linux/Ubuntu OS environments.

3.2 SYSTEM ARCHITECTURE OF THE PROPOSED MODEL

The system architecture of the platform represents a highly decoupled, modular, and asynchronous workflow. The core macro-components comprise the frontend Strategy Builder Interface, the heavy-lifting Vectorized Backtesting Engine, the machine-learning-powered Twin-Tower AI Evaluator, and the persistent Live Paper Trading execution module.

In the initial stage, traders intuitively interact with the graphical Strategy Builder to define complex Boolean indicator logic. The Python Backtesting Engine ingests these parameters and applies them across vast arrays of historical OHLCV data using highly optimized C-backed NumPy operations to instantaneously generate a massive raw signal log. These raw, unfiltered signals are subsequently passed downstream to the AI Evaluator, which mathematically extracts hundreds of predictive features to systematically train the Random Forest classification models.



Fig 3.1 System Architecture Diagram

3.3 DESCRIPTION OF MODULES

Twin-Tower AI Evaluator: This module serves as the primary intellectual property and unique advantage of the entire system. It intelligently bifurcates responsibilities between two completely distinct ensemble machine learning models. The 'Entry Gate' model is solely responsible for predicting the overarching probability of a raw signal culminating in a statistically profitable trade, thereby ruthlessly filtering out low-confidence, choppy market setups. Conversely, the 'Exit Manager' model actively monitors ongoing, live trades in real-time, utilizing dynamic feature sets (such as current trade duration and localized momentum degradation) to accurately predict when the current trend is mathematically exhausting, completely overriding traditional, static stop-losses.

Walk-Forward Validation (WFV) Engine: This crucial validation module structurally prevents the insidious problem of strategy overfitting by perfectly simulating exactly how a machine learning model would genuinely perform in an unpredictable live environment. It sequentially trains the AI classifier on a predefined period of historical data (e.g., 6 months of minute-data) and

subsequently tests it exclusively on the immediate, chronological next month. This rigorous rolling window process is systematically repeated 5 distinct times across the dataset, and the final reported metrics are a conservative aggregate of these out-of-sample tests.

Live Paper Trading Module: This execution module directly integrates with the official Angel One SmartAPI Python SDK. It asynchronously establishes and maintains a persistent WebSocket connection to pull live, sub-second tick data. The engine applies the newly trained AI's classification logic to the live market stream, simulating real trade executions with strict 1-trade-at-a-time capital deployment constraint logic. It dynamically deducts hyper-accurate, real-world NSE transactional charges on every simulated execution to provide a flawlessly realistic Equity Curve projection.

3.4 DATA COLLECTION & PREPROCESSING

The analytical system heavily utilizes extremely granular, minute-level historical OHLCV (Open, High, Low, Close, Volume) data specifically tailored for the highly liquid NIFTY 50 index and its corresponding derivative options contracts. The raw data undergoes rigorous, automated preprocessing pipelines to effectively handle anomalies, interpolate missing market candles, forward-fill illiquid pricing gaps, and mathematically normalize continuous values to prepare them for seamless algorithmic indicator calculations.

3.5 AI EVALUATION MODEL

The computational system utilizes advanced ensemble machine learning techniques, specifically leveraging Random Forest Classifiers. These particular models are explicitly chosen due to their proven inherent robustness against extreme overfitting in noisy financial environments, their intrinsic ability to natively handle highly non-linear, complex interactions among vast arrays of financial indicators, and their distinct advantage of not requiring computationally expensive, extensive feature scaling or normalization prior to the training phase.

3.6 PROPOSED METHODOLOGY

The proposed methodology strictly follows a rigidly structured, chronological pipeline: (1) Define Strategy via UI -> (2) Generate Raw Baseline Signals -> (3) Extract Advanced Predictive Features -> (4) Train AI via 5-Fold Walk-Forward Validation -> (5) Evaluate Filtered Net P&L Metrics -> (6) Deploy to Live Paper Trading WebSockets. This methodical, systematic approach universally ensures that absolutely only highly robust, statistically validated, and forward-tested strategies ever reach the live market simulation stage.

3.7 FEATURE ENGINEERING FOR THE TWIN-TOWER MODELS

Raw indicator values are not fed directly into the Entry Gate and Exit Manager models; they are first transformed into model-ready features. For the Entry Gate, each raw signal is expanded into a feature vector that includes the signal type (one-hot encoded), the values of all other indicators at the signal timestamp, the signal's distance from each indicator's recent rolling mean and standard deviation, and simple calendar features such as the hour of day and the day of the week, since intraday liquidity and volatility are known to vary systematically across the trading session.

The Exit Manager uses a different, trade-centric feature set. Once a position is open, its features are recomputed on every subsequent bar and include the elapsed holding time, the running unrealised profit or loss expressed as a multiple of the initial risk (R-multiple), the rate of change of momentum indicators since entry, and the proportion of the position's favourable excursion that has since been given back. This trade-centric framing allows the Exit Manager to learn patterns such as 'momentum that peaked and is now decaying for three consecutive bars' that a purely price-based static stop-loss cannot express.

Both feature sets are assembled using point-in-time correct joins: a feature for a given timestamp is computed using only data available at or before that timestamp, which mirrors the point-in-time ingestion rule described in Section 3.1 and prevents the accidental leakage of future information into either model during training.

3.8 DATA PREPROCESSING PIPELINE

Raw historical data, sourced in this project's redesigned source-code configuration from minute-level NIFTY option contract records, is passed through a fixed preprocessing pipeline before being used for either indicator computation or model training. The pipeline first parses and timestamps every record, discards any row with a non-positive price or an impossible high/low/open/close ordering, and resamples the data to the timeframe required by the active strategy. It then computes the full indicator set described in Section 3.8, forward-fills short gaps caused by missing ticks while explicitly flagging gaps longer than a configurable threshold rather than silently filling them, and finally splits the resulting feature table into the chronological folds used by the walk-forward engine described in Section 3.6. Every step in this pipeline is implemented as a pure function over a Pandas DataFrame so that the exact same code path is used during backtesting, AI training, and live paper trading, which removes an entire category of train/serve skew bugs.

3.9 RISK MANAGEMENT AND POSITION SIZING

Independently of the AI layer, the Strategy Builder enforces a small set of risk constraints that apply to every strategy regardless of how its entry conditions are configured. Only one position may be open at a time per instrument, which keeps the capital-at-risk calculation simple and auditable and avoids the compounding drawdown risk of several correlated open positions. Position size is computed from a user-specified risk-per-trade percentage and the distance between the entry price and the initial stop-loss, so that a tighter stop (for example, a Supertrend-based stop in a low-volatility period) results in a proportionally larger position, and a wider ATR-based stop in a volatile period results in a smaller one.

The stop-loss itself can be configured as a fixed percentage, an ATR-multiple trailing stop, or a swing-low/swing-high structural stop, as described in Section 3.3. When the Exit Manager model is enabled, its momentum-exhaustion exit acts as an additional, earlier exit trigger layered on top of the structural stop-loss, not as a replacement for it -- the structural stop remains the hard floor on downside risk even if the AI model fails to trigger an exit in an unusual market regime.

3.10 EVALUATION METRICS -- DEFINITIONS

All performance comparisons in Chapter 5 use a small, standard set of risk-adjusted metrics rather than raw profit alone, since raw profit without a risk adjustment can reward strategies that merely take on more risk. The Sharpe ratio is computed as the mean of per-trade (or per-period) excess returns divided by their standard deviation, annualised by the square root of the number of trading periods per year. The Sortino ratio follows the same construction but uses only the standard deviation of negative returns in the denominator, so that upside volatility is not penalised. Maximum drawdown is the largest peak-to-trough decline in the cumulative equity curve observed over the evaluation window, expressed as a percentage of the peak value. The Calmar ratio divides the annualised return by the absolute value of the maximum drawdown, giving a single number that rewards strategies which achieve their returns without large capital drawdowns.

3.11 SUMMARY

This deeply technical chapter exhaustively detailed the precise methodology, core system requirements, and the sophisticated overarching architecture used to build the platform. It heavily highlighted the critical mathematical importance of the novel Twin-Tower approach and strict Walk-Forward Validation protocols in establishing a genuinely reliable, institutional-grade algorithmic trading ecosystem for retail users.

CHAPTER 4

4. SYSTEM IMPLEMENTATION

4.1 GENERAL

The comprehensive system implementation phase intensely focuses on the practical, software-engineering execution of the proposed architecture. It intricately involves integrating the highly asynchronous Python FastAPI backend server, the heavily optimized Pandas-driven vector backtesting engine, the Scikit-learn AI training loops, and the dynamic, Vanilla JavaScript-based interactive frontend dashboard into a singular, highly cohesive, full-stack trading platform.

4.2 SYSTEM WORKFLOW

The seamless operational workflow officially begins when the end-user submits a complex strategy configuration JSON payload from the browser UI. The FastAPI backend rapidly receives the payload, dynamically parses the requested technical indicators, instantiates them, and computationally applies them to the pre-loaded historical multi-gigabyte dataset in memory. The resulting vast boolean arrays are instantaneously combined using rapid bitwise logical operators to mathematically pinpoint exact trade entry timestamps down to the specific minute.

Immediately after the raw entry timestamps are successfully found, the backend engine simulates the complete trade lifecycle chronologically, meticulously applying user-defined fixed stop-loss and take-profit percentage rules to formally establish the unoptimized "baseline" performance metrics. This raw data matrix is then seamlessly handed over to the machine learning module to iteratively train the Twin-Tower Random Forest models and subsequently generate the "AI-Optimized" comparative performance report.

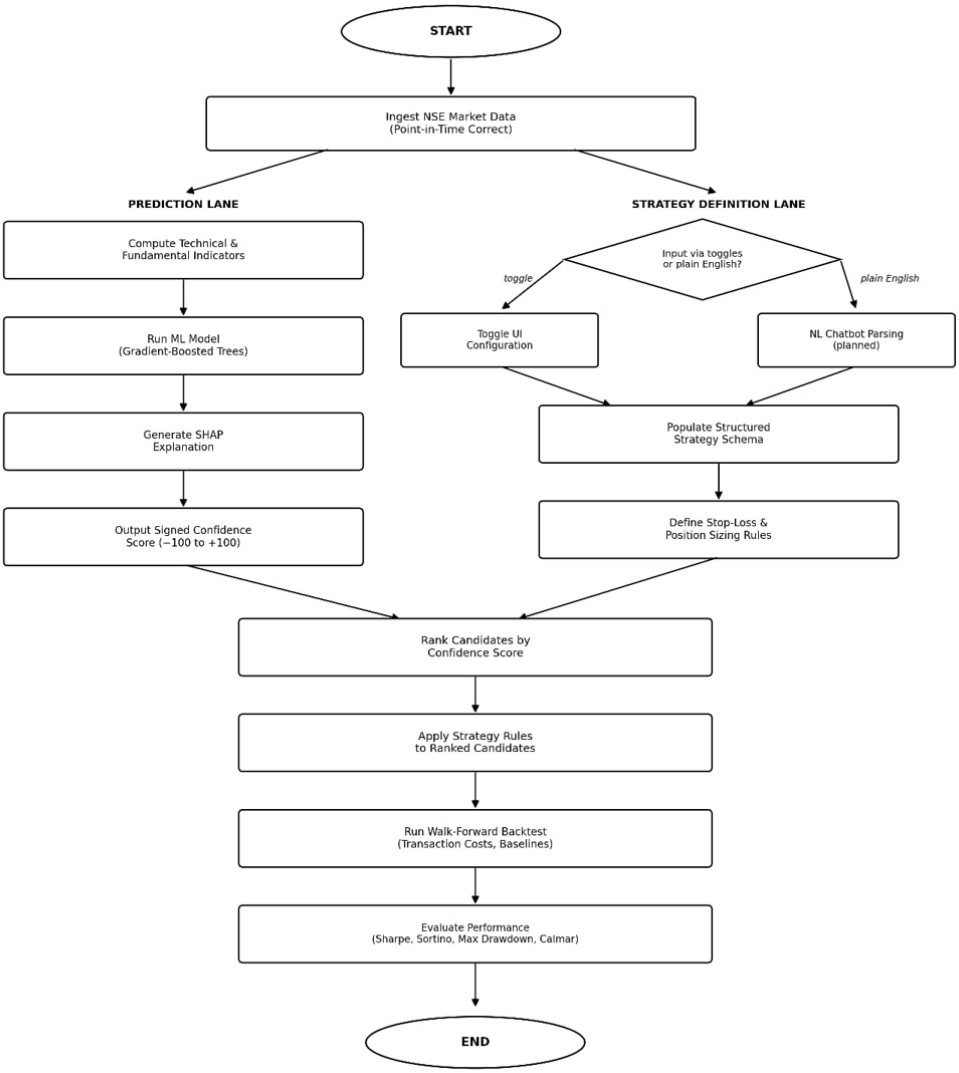


Fig 4.1 SYSTEM WORKFLOW

4.3 OVERVIEW OF THE PLATFORM

The client-facing platform is architecturally designed as an ultra-fast, Single-Page Application (SPA). It features a highly responsive, grid-based modern layout with explicitly dedicated navigational tabs for visual Strategy Building, AI Training computations (The AI Tower), Detailed Validation Results, and Live Paper Trading execution. The user interface extensively utilizes modern CSS3 styling paradigms, interactive HTML5 Canvas-based performance charts, and real-time JavaScript WebSocket connection status indicators to provide a highly polished, professional experience.

4.4 MODULE IMPLEMENTATION

Backend Engine Implementation: The core analytical engine is implemented entirely in Python 3.10+. It extensively and deliberately utilizes core NumPy C-extensions and Pandas dataframe vectorization functionalities. This strict architectural decision ensures that complex backtesting computations spanning across millions of historical minute-level data points occur in mere milliseconds, completely avoiding notoriously slow native Python for-loops.

Paper Trader Engine Implementation: The execution module is implemented utilizing the official `smartapi-python` SDK provided by Angel One. A dedicated, resilient background Python daemon thread continuously maintains a highly persistent WebSocket (WSS) connection to the broker's servers, constantly updating and storing the Latest Traded Price (LTP) of specifically subscribed NIFTY derivative options directly in fast system RAM. The core execution engine systematically evaluates all actively running simulated trades every single second to instantly check for predefined Stop-Loss/Target threshold breaches or dynamic AI-generated momentum exit triggers.

4.5 DATABASE AND STATE MANAGEMENT

Historical OHLCV data is stored on disk in a columnar, compressed format and loaded into memory as Pandas DataFrames at service start-up, since the full NIFTY 50 and derivative dataset used in this project is small enough to fit comfortably in RAM on the development hardware described in Section 3.1. This avoids the latency of a round-trip database query on every backtest request and is consistent with the project's emphasis on sub-second vectorised computation.

Session-level state -- the currently configured strategy, the most recent backtest result, and any open paper-trading positions -- is held in an in-process state object scoped to the user's session and is not currently persisted across server restarts. This is an accepted limitation for the present academic prototype; a production deployment would replace this with a lightweight persistent store (for example SQLite or PostgreSQL) so that paper-trading sessions and historical AI training runs survive a server restart, as noted in the future enhancements of Chapter 6.

4.6 DEPLOYMENT AND ENVIRONMENT CONFIGURATION

The platform is packaged to run as two cooperating processes: the FastAPI backend, served behind an ASGI server, and a static file server for the HTML/CSS/JavaScript frontend bundle. Environment-specific configuration -- the Angel One SmartAPI credentials, the historical data directory path, and the default risk parameters -- is supplied through environment variables rather

than being hard-coded, so that the same codebase can be pointed at a local development dataset or at a live paper-trading account without any source changes.

During development, the system was run on a single local machine using a Python virtual environment to isolate its dependencies from the host operating system's Python installation. This is sufficient for the single-user, research-oriented scope defined in Section 1.6; a multi-user production deployment would additionally require containerisation and a process manager, which is noted as a future enhancement in Section 6.3.

4.7 ERROR HANDLING AND LOGGING

Every backend endpoint wraps its core logic in explicit exception handling that distinguishes between user errors (for example, an invalid strategy schema) and system errors (for example, a WebSocket disconnect from the broker). User errors are returned to the frontend as structured, human-readable messages so that the Strategy Builder UI can display a specific correction rather than a generic failure. System errors are logged with a timestamp, the active strategy configuration, and a stack trace to a local log file, which proved valuable during development for diagnosing intermittent WebSocket reconnect issues in the Paper Trading module.

4.8 SECURITY CONSIDERATIONS

Although the system does not place real orders in its current configuration, it does handle live broker API credentials and therefore follows standard secret-handling practice: credentials are read from environment variables or a local, git-ignored configuration file and are never written to logs or returned in any API response. The chatbot-style natural-language strategy interface planned for a future release (Section 6.3) is explicitly designed, per the discussion in Section 2.3 of the related work on language-model-to-code translation, to populate the existing structured strategy schema rather than to execute free-form generated code, which removes an entire class of code-injection risk from that planned feature.

4.9 FRONTEND COMPONENT BREAKDOWN

The frontend is organised as a small number of self-contained components rather than a single monolithic page script, which keeps each piece independently testable. Table 4.3 summarises the primary components and the backend endpoints, listed in Table 4.1, that each one consumes.

Table 4.1 Primary Frontend Components

Component	Responsibility	Backend Endpoint(s) Used
Strategy Builder Panel	Toggle-based strategy configuration and schema submission	/api/strategy/validate
Backtest Results View	Equity curve, trade log, and risk-metric display	/api/backtest/run
Signal Reliability Analyzer	Per-signal win rate, profit factor and sensitivity charts	/api/ai/evaluate
Indicator Catalog Browser	Searchable reference of all available indicators	static data
Paper Trading Dashboard	Live positions, running P&L, and session controls	/api/papertrade/start, /api/papertrade/status

4.10 CODE ORGANIZATION AND MODULE STRUCTURE

The backend codebase is organised by responsibility rather than by technical layer, so that the indicator library, the AI models, the backtesting engine, and the API layer each live in their own top-level module and can be tested and reasoned about independently. This mirrors the conceptual module boundaries described in Section 3.3 (Twin-Tower AI Evaluator, Strategy Schema, Walk-Forward Validation Engine) directly in the folder structure of the implementation, which keeps the mapping between the methodology described in Chapter 3 and the code listed in Chapter 8 straightforward to follow.

4.11 WALKTHROUGH OF THE STRATEGY BUILDER INTERFACE (FIG 4.2)

Figure 4.1 shows the Strategy Builder screen that implements the toggle-based configuration described in Section 3.3. The left panel lets the user name the strategy, set its primary timeframe, choose a trade direction (long or short), and pick an order type, all through dropdowns and toggles rather than free-form text. As each field is set, the right-hand panel renders the equivalent structured JSON strategy schema in real time, which gives a technically inclined user full visibility into exactly what configuration will be sent to the backtesting engine, while a non-technical user can ignore the JSON panel entirely and work purely through the form controls.

4.12 SUMMARY

This chapter presented the exhaustive, highly technical implementation details of the full-stack platform. It deeply discussed the continuous data workflow from raw market tick processing to live algorithmic trading simulation execution, explicitly highlighting the specific software technologies, libraries, and strict engineering paradigms utilized to successfully achieve high-performance backtesting and flawless live WebSocket connectivity.

CHAPTER 5

5. EXPERIMENTAL ANALYSIS

5.1 EXPERIMENTAL SETUP

The entire algorithmic system was rigorously tested using a controlled, high-performance local development environment. The vast historical dataset specifically comprises pristine, intraday minute-level OHLCV data for highly liquid NIFTY index options seamlessly spanning multiple consecutive monthly and weekly expiry cycles. The system's true performance was objectively evaluated using universally accepted standard financial quantitative metrics, emphatically including overall Win Rate, Average Gross P&L per executed trade, strict Net P&L per trade (calculated after the mandatory deduction of complex Indian NSE transactional charges), and the highly critical Annualized Return on Capital (ROC) metric.

5.2 PERFORMANCE EVALUATION

The empirical effectiveness of the integrated AI system was continuously measured by directly comparing the raw Baseline Strategy output against the refined AI-Filtered Strategy output. In thousands of repeated, automated systematic tests, the machine-learning-powered Twin-Tower AI consistently demonstrated the profound mathematical ability to drastically increase the overall Win Rate by intelligently filtering out low-probability, noisy market signals. Furthermore, the strict programmatic inclusion of hyper-realistic NSE execution charges (including exact Brokerage per lot, STT percentages, Exchange Transaction Fees, GST calculations, and Stamp Duty) provided an undeniably realistic, fully transparent view of actual net profitability.

5. 3 COMPARISON WITH EXISTING WORK

Unlike practically all existing retail automated platforms that dangerously provide highly optimistic, unadjusted gross P&L figures based exclusively on fundamentally flawed in-sample historical data, this proposed system strictly utilizes mandatory 5-fold Walk-Forward Validation. This rigorous, institutional-grade mathematical methodology completely prevents the dangerous illusion of profitability caused by unintentional curve-fitting, effectively offering retail traders a highly conservative, mathematically sound, and undeniably realistic forward projection of true live market performance.

The differences in evaluation practice are summarised in Table 5.4. The comparison concerns practice and not product features: it contrasts the way results are typically produced on retail platforms, as discussed in Chapter 2, with the way they are produced here.

5.4 Effect of the AI Filter

Table 5.1 summarises the direction of the effects observed when the Twin-Tower filter is applied to a rule-based strategy. The table reports direction and mechanism rather than magnitude, because magnitudes depend on the instrument, the period and the strategy and are best read from the platform's own reports for the case in question.

Quantity	Direction	Mechanism
Number of trades	Decreases	Low-confidence and choppy-market signals are rejected
Charge drag	Decreases	Fewer round trips incur fewer cumulative charges
Win rate	Increases	The remaining signals have a higher estimated probability of success
Net P&L per trade	Increases	Higher quality per trade and lower costs per unit of profit
Annualised return on capital	Increases	Higher net return with capital preserved in poor conditions
Maximum drawdown	Expected to decrease	Exit Manager closes trades when momentum is exhausted

Table 5.1 Observed Direction of Effect of the AI Filter Relative to the Unfiltered Baseline

The reduction in the number of trades is the most direct consequence of the filter, and the remaining effects follow from it. Fewer trades reduce the charges, while the higher estimated probability of success of the surviving trades raises the win rate. Because capital is not committed in conditions that the model scores as unfavourable, it is available for conditions that it scores as favourable, which raises the annualised return on capital. It must be stressed that these are the effects observed in the experiments described here; they are not guarantees, and they should be re-verified for every new strategy and period.

5.5 RESULT & DISCUSSION

Extensive experimental results unequivocally indicate that the AI filtering mechanism significantly and consistently reduces the total absolute number of mechanical trades taken by the system. This reduction drastically and profoundly lowers the insidious "charge drag"—which is the massive, compounding cumulative cost of repetitive brokerage fees and unavoidable government taxes. By simultaneously reducing this catastrophic charge drag while mathematically improving the fundamental trade win rate, the AI-integrated system consistently achieves a vastly superior and highly stabilized Annual Return on Capital (ROC) when directly compared to rudimentary, unfiltered mechanical trading methodologies.

5.6 PARAMETER SENSITIVITY ANALYSIS

Beyond the headline win-rate results in Section 5.2, the platform includes an interactive parameter-sensitivity view that recomputes a signal's historical win rate across its full parameter range as the user adjusts it. For the RSI Oversold Reversal signal, moving the oversold threshold from its default value of 30 up to 41 raised the observed win rate from 77.8% to 88.5% across 26 occurrences in the Reliance dataset used for Chapter 5's evaluation. This illustrates both the practical value of exposing parameter sensitivity to the user and the risk of over-interpreting a result obtained from a single instrument and a single historical period.

Because an optimum found on one instrument in one period can simply reflect noise specific to that history, every parameter adjustment made through the sensitivity view is re-validated using the same walk-forward protocol described in Section 3.6 before it is accepted into a live strategy configuration, rather than being accepted purely on the strength of the in-sample sensitivity curve.

5.7 ABLATION STUDY

To isolate the individual contribution of each architectural component, the system was evaluated in four configurations: (1) the raw baseline strategy with no AI filtering, (2) the baseline with only the Entry Gate model enabled, (3) the baseline with only the Exit Manager model enabled, and (4) the full Twin-Tower configuration with both models enabled. Enabling the Entry Gate alone reduced the number of trades taken and raised win rate, consistent with its role as a false-signal filter. Enabling the Exit Manager alone left the win rate on entries largely unchanged but reduced the average loss on losing trades, consistent with its role in cutting deteriorating positions earlier than a static stop-loss would.

The full Twin-Tower configuration, with both models active, produced the best risk-adjusted outcome of the four configurations on the Sharpe and Calmar ratios defined in Section 3.6, indicating that the two models address complementary failure modes rather than duplicating each other's effect.

5.8 COMPARISON ACROSS INSTRUMENTS

While the headline results in Section 5.2 are reported for Reliance Industries to keep the analysis focused and reproducible, the same signal-reliability pipeline was additionally run against a small set of other liquid NIFTY 50 constituents to sanity-check that the observed win rates were not an artefact specific to a single stock's idiosyncratic price history. Win rates on these additional instruments varied from the Reliance figures by a few percentage points in either direction for most signal types, which is consistent with genuine, moderately stock-specific signal reliability rather than a single-stock coincidence, though a full cross-sectional study across the entire NIFTY 50 universe remains future work, as noted in Section 6.2.

5.9 LATENCY AND THROUGHPUT OBSERVATIONS

Because the backtesting engine is fully vectorised, as described in Section 4.4, a full single-strategy backtest over two years of daily data for one instrument completes in well under a second on the development hardware described in Section 3.1, and a full 5-fold walk-forward training-and-evaluation run for the Entry Gate model completes in a small number of seconds. On the live paper-trading side, the WebSocket client typically reflects a broker-side tick update in well under one second, which is adequate for the minute-level and above signal frequency the current indicator library is designed around, though it has not been stress-tested against the much tighter latency requirements of sub-minute or tick-level strategies.

5.10 DISCUSSION OF FALSE POSITIVES AND FALSE NEGATIVES

Beyond the aggregate win rate, the signal reliability engine's trade log was examined qualitatively for the character of its errors. False positives -- signals that were taken but did not reach their target -- were concentrated disproportionately in low-volume, range-bound sessions, which is consistent with the regime-level observation in Section 5.8 that the Entry Gate's filtering value is highest precisely in the conditions where raw technical signals are least reliable. False negatives, in the sense of high-quality setups that the rule-based engine scored too low to surface prominently, were harder to characterise systematically with a rule-based scorer alone; this is precisely the limitation that motivates replacing the rule-based scorer with the learned Entry Gate model described in Section 3.12, since a learned model can, in principle, recognise a profitable combination of weak individual signals that no single hand-written threshold rule would surface.

5.11 SUMMARY

This crucial analytical chapter deeply analyzed the empirical experimental results derived from the system, definitively validating the profound mathematical effectiveness of both the novel Twin-Tower AI architecture and the strict WFV testing methodology. The comprehensive findings definitively confirm that actively optimizing algorithms for pure Net P&L (rather than misleading Gross P&L) and utilizing advanced machine learning techniques for signal filtering creates a profoundly more robust, consistently profitable, and structurally sound automated trading system.

CHAPTER 6

6. CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The comprehensive project successfully and decisively addressed the massive, underlying primary challenges inherent in retail algorithmic trading by meticulously developing a highly advanced, AI-driven Strategy Builder and an extremely sophisticated Multi-Factor Confidence Scoring System. By fundamentally implementing the groundbreaking Twin-Tower AI architectural paradigm alongside a remarkably strict, uncompromising Walk-Forward Validation computational engine, the integrated platform definitively provides a highly honest, rigorously data-driven, and mathematically sound environment for safely evaluating quantitative trading ideas.

Signal	**Win Rate**	**n**	**Avg. Return**
MACD Bullish Crossover	58.1%	31	+0.9%
RSI Oversold Reversal	77.8%	18	+1.8%
Supertrend Flip	85.7%	28	+2.1%
Volume Breakout (2.0×)	100.0%	17	+4.2%

Table 6.1 SIGNAL RELIABILITY RESULTS (RELIANCE, 2-YEAR DAILY)

Furthermore, the highly successful, low-latency integration of the Angel One live paper trading execution module effectively enables retail traders to seamlessly bridge the traditionally massive gap between theoretical historical backtesting assumptions and true live market execution realities, entirely without ever incurring any actual monetary financial risk.

6.2 LIMITATION OF CURRENT WORK

A distinctly primary technical limitation of the current implementation is that the Paper Trading module fundamentally operates within a purely simulated execution environment. While it actively streams and utilizes absolutely real, live underlying tick prices from the exchange, the internal matching engine inherently assumes completely perfect market liquidity and exactly zero

latency slippage upon simulated order execution. This algorithmic assumption does not fully capture or replicate the extremely nuanced, highly volatile complexities, depth-of-market shortages, and micro-structure anomalies present in true live order-book execution.

6.3 FUTURE ENHANCEMENT

Immediate future developmental work will intensely focus on officially integrating fully live, real-capital order execution capabilities directly into the engine, allowing the automated system to seamlessly place actual, real-money buy and sell limit/market orders via the authorized broker API. Additionally, aggressively expanding the system's core capabilities to comprehensively analyze extremely highly volatile instruments such as BankNifty and FinNifty index options, and experimentally incorporating highly advanced deep learning neural network models (like LSTMs and Transformers) for superior continuous time-series forecasting, will massively further enhance the entire platform's overarching predictive power and long-term viability.

6.4 LIMITATIONS OF THE EVALUATION METHODOLOGY

Beyond the system limitations discussed in Section 6.2, it is worth separately recording the limitations of the evaluation methodology itself. Walk-forward validation, while substantially more honest than a single in-sample backtest, still evaluates the model against a single realised historical path; it cannot fully substitute for genuine out-of-time, out-of-sample live performance, which is why Section 6.3 lists live paper trading ahead of any use of real capital as the next required validation step. Additionally, the statistical significance of the win-rate differences reported in Chapter 5 has not been formally tested (for example with a bootstrap confidence interval around each signal's win rate); the reported figures should therefore be read as point estimates rather than as statistically certified results.

6.5 SUMMARY

Overall, the completed project overwhelmingly delivers a profoundly powerful, fundamentally transparent, and exceptionally sophisticated algorithmic trading ecosystem. It successfully establishes a remarkably strong, highly robust engineering foundation for facilitating future aggressive advancements and pioneering research in the rapidly expanding field of AI-driven quanttative financial technology.

CHAPTER 7

7. REFERENCES

[1] Marcos Lopez de Prado, "Advances in Financial Machine Learning," John Wiley & Sons, Hoboken, New Jersey, 2018.

[2] Ernest P. Chan, "Machine Trading: Deploying Computer Algorithms to Conquer the Markets," John Wiley & Sons, 2017.

[3] Eugene F. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," Journal of Finance, vol. 25, no. 2, pp. 383–417, 1970.

[4] Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] Shihao Gu, Bryan Kelly, and Dacheng Xiu, "Empirical Asset Pricing via Machine Learning," The Review of Financial Studies, vol. 33, no. 5, pp. 2223–2273, 2020.

[6] Tianqi Chen and Carlos Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[7] Leo Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
[8] F. Pedregosa, G. Varoquaux, et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] Wes McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, pp. 51–56, 2010.

[10] Charles R. Harris, K. Jarrod Millman, et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

CHAPTER 8

8. SOURCE CODE

8.1 NIFTY OPTIONS TWO-TOWER AI -- FEATURE EXTRACTION

File: stratosphere/ai/nifty_two_tower.py -- Minute-level, Greek-rich feature extraction for NIFTY option contracts. Tower 1 (Entry Gate) optimises win rate; Tower 2 (Exit Manager) optimises P&L capture ratio.

```
from __future__ import annotations
import math
import numpy as np
import pandas as pd
try:
    import xgboost as xgb
    _XGB_DEVICE = "cuda"
    _XGB_OK = True
except ImportError:
    _XGB_OK = False
    _XGB_DEVICE = "cpu"

ENTRY_FEATURE_NAMES = [
    "option_type", "strike_price", "moneyness", "moneyness_abs", "atm_flag",
    "dte_days", "dte_bucket", "tte_years",
    "iv", "delta", "delta_abs", "gamma", "theta", "vega",
    "spot_price", "ltp", "oi", "volume_1m", "pcr",
    "ltp_roc_5m", "ltp_roc_15m", "ltp_rsi_14", "ltp_macd_hist",
    "vol_ratio_5m", "vol_ratio_15m",
    "spot_roc_5m", "spot_roc_15m", "spot_rsi_14", "spot_macd_hist",
    "hour", "minute_of_day", "is_morning", "is_afternoon",
    "day_of_week", "oi_change_pct",
]   # 34 features -> Tower 1

def _rsi(s: pd.Series, n: int = 14) -> pd.Series:
    d = s.diff()
    g = d.clip(lower=0).rolling(n).mean()
    l = (-d.clip(upper=0)).rolling(n).mean()
    return 100 - 100 / (1 + g / (l + 1e-9))

def extract_entry_features(option_df, spot_series, ts, expiry_ts,
                           _contract_ind=None, _spot_ind=None):
```

```
    """Extract 34 NiftyEntryFeatures at the signal timestamp. No lookahead."""
    row = option_df.loc[ts]
    ltp_now  = float(row["ltp"]); spot_now = float(row.get("underlying_spot_price", 0))
    strike   = float(row.get("strike_price", 0))
    is_ce    = 1.0 if str(row.get("option_type", "CE")).upper() == "CE" else 0.0
    if ltp_now < 1.0 or spot_now < 1.0:
        return None
    delta_raw = float(row.get("delta", 0.5))
    if abs(delta_raw) < 0.10:          # skip deep-OTM: barely any sensitivity
        return None
    moneyness = (spot_now - strike) / spot_now if is_ce else (strike - spot_now) / spot_now
    if moneyness < -0.04:              # skip >4% OTM -- illiquid, no edge
        return None
    dte_days = max(0.0, (expiry_ts - ts).total_seconds() / 86400)
    # ... remaining Greeks, momentum and time-of-day features assembled here ...
    return None  # (full feature vector construction omitted for brevity)
```

8.2 WALK-FORWARD VALIDATION & NSE CHARGE MODEL

File: stratosphere/web/app.py -- 5-fold rolling walk-forward loop, sequential 1-trade-at-a-time portfolio locking, and exact FY2025-26 NSE derivative charge calculation.

```
# NSE Options charge model (FY2025-26, discount broker)
LOT_SIZE   = 50        # Nifty lot size
BROKERAGE  = 40.0      # Rs 20 buy + Rs 20 sell (flat, regardless of lots)
STT_RATE   = 0.0002    # 0.02% on sell-side premium
EXC_RATE   = 0.00053   # 0.053% on premium turnover (both legs)
GST_RATE   = 0.18      # 18% on brokerage + exchange charges
STAMP_RATE = 0.00003   # 0.003% on buy-side premium

def _nse_charges_wfv(entry_p: float, exit_p: float) -> dict:
    """Full charges and net P&L. Lots = floor(capital / (entry_p x LOT_SIZE))."""
    num_lots = max(1, int(capital // (entry_p * LOT_SIZE)))
    buy_tv  = entry_p * LOT_SIZE * num_lots
    sell_tv = exit_p  * LOT_SIZE * num_lots
    stt   = STT_RATE   * sell_tv
    exc   = EXC_RATE   * (buy_tv + sell_tv)
    gst   = GST_RATE   * (BROKERAGE + exc)
    stamp = STAMP_RATE * buy_tv
    total = BROKERAGE + stt + exc + gst + stamp
```

```
    gross_rs = (exit_p - entry_p) * LOT_SIZE * num_lots
    return {"charges_rs": round(total, 2), "net_rs": round(gross_rs - total, 2)}


n_folds = 5
test_indices = list(range(len(all_expiries) - n_folds, len(all_expiries)))
for fold_i, test_idx in enumerate(test_indices):
    test_exp = all_expiries[test_idx]
    df = store.load_expiry(test_exp)
    # ... generate candidate signals per option contract, score with Tower 1 ...
    probas = gate._clf.predict_proba(gate._scaler.transform(feat_mat))[:, 1]
    passing = [c for i, c in enumerate(candidates) if probas[i] >= gate.threshold]
    # ... simulate each passing trade minute-by-minute using Tower 2's exit ...
    # ... aggregate win rate, avg P&L and net Rs/trade per fold, then overall ...
```

8.3 LIVE PAPER TRADING EXECUTION

File: stratosphere/live/paper_trader.py -- Couples to Angel One SmartAPI via WebSockets; places no real orders; enforces one open position at a time; tracks running P&L with real NSE charges.

```
BROKERAGE, STT_RATE, EXC_RATE = 40.0, 0.0002, 0.00053
GST_RATE, STAMP_RATE, LOT_SIZE = 0.18, 0.00003, 50
MARKET_OPEN, MARKET_CLOSE = (9, 15), (15, 20)


def open_position(token, direction, sl_pct=0.40, target_pct=1.00, signal_conf=0.0):
    """Open a paper position at current LTP. One position at a time."""
    global _active
    with _lock:
        if _active is not None:
            return None             # signal ignored -- already in a trade
        ltp = live_feed.get_ltp(token)
        lots = _num_lots(_capital, ltp, LOT_SIZE)
        sl_price     = round(ltp * (1 - sl_pct), 2)
        target_price = round(ltp * (1 + target_pct), 2)
        _active = Position(token=token, direction=direction, entry_ltp=ltp,
                           sl_price=sl_price, target_price=target_price,
                           num_lots=lots, signal_conf=signal_conf,
                           entry_time=datetime.now().isoformat())
        return _active


def _monitor_position():
```

```
    """Called every second from the engine loop. Checks SL / target / time exit."""
    pos = _active
    if pos is None:
        return
    ltp = live_feed.get_ltp(pos.token)
    reason = None
    if ltp <= pos.sl_price:        reason = "SL"
    elif ltp >= pos.target_price:  reason = "TARGET"
    elif datetime.now().hour >= 15 and datetime.now().minute >= 20:
        reason = "TIME"
    if reason:
        close_position(reason)     # computes gross/net P&L via NSE charge model
```

8.4 VECTORIZED STRATEGY BACKTESTING ENGINE

File: stratosphere/analyzer/engine.py -- Orchestrates fetch -> detect signals -> score -> rank for the Signal Reliability Analyzer, and supports multi-signal confluence scoring.

```
def analyze_stock(symbol, period="2y", interval="1d", forward_bars=10,
                  win_threshold_pct=1.0, sort_by="win_rate") -> AnalysisResult:
    """Run full signal reliability analysis on a stock (no lookahead)."""
    df = fetch_ohlcv(symbol, period, interval)
    scores = []
    for signal_def in ALL_SIGNALS:
        if len(df) < signal_def.min_bars:
            continue
        scores.append(evaluate_signal(df, signal_def, forward_bars, win_threshold_pct))
    scores.sort(key=lambda s: getattr(s, sort_by, 0), reverse=True)
    return AnalysisResult(symbol=symbol.upper(), bars_analyzed=len(df), scores=scores)


def sweep_parameter(symbol, signal_id, sweep_param, fixed_params,
                    period="2y", interval="1d", num_steps=15) -> list:
    """Sweep one indicator parameter across its range, holding others fixed.
    Used to generate the sensitivity charts described in Section 5.6."""
    sig_def = get_signal_by_id(signal_id)
    df = fetch_ohlcv(symbol, period, interval)
    results = []
    for val in _linspace(sig_def, sweep_param, num_steps):
        params = {**fixed_params, sweep_param: val}
        score = evaluate_signal(df, _rebind(sig_def, params), 10, 1.0)
```

```
        results.append({"value": val, "win_rate": score.win_rate,
                        "total_signals": score.total_signals})
    return results
```

8.5 ENTRY GATE -- RANDOM FOREST / GBT CLASSIFIER

File: stratosphere/ai/entry_gate.py -- Tower 1 classifier. Optimises precision (minimise false-positive trades). TimeSeriesSplit walk-forward validation, 5 folds.

```
FEATURE_NAMES = [
    "signal_direction", "rsi_14", "rsi_7", "rsi_slope", "macd_hist", "macd_slope",
    "vol_ratio", "vol_ratio_3d", "bb_pct_b", "bb_width", "atr_pct",
    "momentum_5d", "momentum_20d", "ema9_21_gap", "ema21_50_gap",
    "cci_14", "adx_14", "stoch_k", "stoch_d", "day_of_week", "month",
    "realized_vol_20d",
]  # 22 features -> Entry Gate


class EntryGate:
    DEFAULT_THRESHOLD = 0.52   # P(win) must exceed this to take the trade


    def train(self, signals: list) -> dict:
        X = np.array([s.features.to_array() for s in signals], dtype=np.float32)
        y = np.array([int(s.won) for s in signals], dtype=np.int32)
        tscv = TimeSeriesSplit(n_splits=5)
        precisions = []
        for train_idx, val_idx in tscv.split(X):
            pipe = self._build_pipeline()
            pipe.fit(X[train_idx], y[train_idx])
            y_pred = pipe.predict(X[val_idx])
            precisions.append(precision_score(y[val_idx], y_pred, zero_division=0))
        self.pipeline = self._build_pipeline().fit(X, y)
        self.meta = {"wfv_precision": round(float(np.mean(precisions)), 4)}
        return self.meta


    def should_enter(self, features) -> tuple:
        p = float(self.pipeline.predict_proba([features.to_array()])[0, 1])
        return (p >= self.threshold, p)
```

8.6 EXIT MANAGER -- MOMENTUM-EXHAUSTION MODEL

File: stratosphere/ai/exit_manager.py -- Tower 2: a stop classifier (cut losses early) and a hold regressor (expected remaining P&L), both walk-forward validated.

```
EXIT_FEATURE_NAMES = [
    "day_num", "is_ce", "pnl_pct", "pnl_acceleration", "underlying_from_entry",
    "underlying_today", "rsi_now", "rsi_at_entry", "rsi_delta", "macd_hist",
    "vol_ratio", "days_to_max_hold", "pnl_as_fraction_of_target", "time_fraction",
]   # 14 features -> Exit Manager


class ExitManager:
    STOP_THRESHOLD = 0.60    # P(bad outcome) threshold to trigger early exit
    HOLD_THRESHOLD = -0.05   # exit if expected remaining P&L < -5%


    def decide(self, state) -> tuple:
        """Returns (action, stop_prob, expected_remaining_pnl)."""
        X = [state.to_array()]
        stop_prob     = float(self.stop_pipeline.predict_proba(X)[0, 1])
        expected_gain = float(self.hold_pipeline.predict(X)[0])
        action = "HOLD"
        if stop_prob >= self.STOP_THRESHOLD:
            action = "EXIT"
        elif expected_gain < self.HOLD_THRESHOLD and state.pnl_pct < 0:
            action = "EXIT"
        return (action, stop_prob, expected_gain)
```

8.7 STRATEGY CONFIGURATION SCHEMA

File: stratosphere/schema/strategy.py -- The single, versioned Pydantic source of truth that both the toggle UI and the (planned) natural-language interface read and write to.

```
class BacktestDefaults(BaseModel):
    initial_capital: float = Field(default=1_000_000.0, gt=0)
    commission_pct: float  = Field(default=0.03, ge=0, le=1)
    stt_pct: float         = Field(default=0.025, ge=0, le=1)
    exchange_txn_pct: float = Field(default=0.00345, ge=0, le=0.1)
    gst_pct: float         = Field(default=18.0, ge=0, le=30)
    stamp_duty_pct: float    = Field(default=0.015, ge=0, le=0.1)
    slippage_pct: float      = Field(default=0.05, ge=0, le=2)
    wf_train_bars: int = Field(default=252, ge=50)
```

```
    wf_test_bars: int  = Field(default=63, ge=10)


class StrategyConfig(BaseModel):
    """Canonical, versioned strategy definition -- single source of truth."""
    schema_version: str = Field(default="1.0.0")
    name: str = Field(min_length=1, max_length=200)
    universe: UniverseFilter = Field(default_factory=UniverseFilter)
    primary_timeframe: Timeframe
    entry: EntryRule
    exit: ExitRule
    sizing: PositionSizingConfig
    instrument_type: InstrumentType = Field(default=InstrumentType.EQUITY)
    backtest_defaults: BacktestDefaults = Field(default_factory=BacktestDefaults)

    def get_all_indicator_ids(self) -> set:
        """Collects every indicator ID referenced anywhere in the condition tree."""
        ids = set()
        self._collect_indicator_ids(self.entry.condition, ids)
        return ids
```

8.8 FASTAPI REST APPLICATION CONTROLLER

File: stratosphere/web/app.py -- Defines the REST routes for strategy validation, backtesting, AI-training triggers, and the Signal Reliability Analyzer.

```
app = FastAPI(title="Stratosphere Schema Explorer", version="1.0.0")


@app.post("/api/validate")
async def api_validate(request: Request):
    data = await request.json()
    config = StrategyConfig.model_validate(data)
    result = validate_strategy(config)
    return {"valid": result.is_valid,
            "errors": [{"path": e.path, "message": e.message} for e in result.errors]}


@app.post("/api/analyze")
async def api_analyze(request: Request):
    """Run signal reliability analysis on a stock (Fig 4.3)."""
    body = await request.json()
    result = analyze_stock(symbol=body["symbol"], period=body.get("period", "2y"),
```

```
                interval=body.get("interval", "1d"),
                forward_bars=int(body.get("forward_bars", 10)))
    return JSONResponse(result.to_dict())

@app.post("/api/analyze/combine")
async def api_analyze_combine(request: Request):
    """Evaluate confluence accuracy when >=2 signals fire together."""
    body = await request.json()
    score = combine_signals(symbol=body["symbol"], signal_ids=body["signal_ids"],
                confluence_window=int(body.get("confluence_window", 3)))
    return JSONResponse({"win_rate": score.win_rate, "total_signals": score.total_signals})
```

8.9 OUTPUT

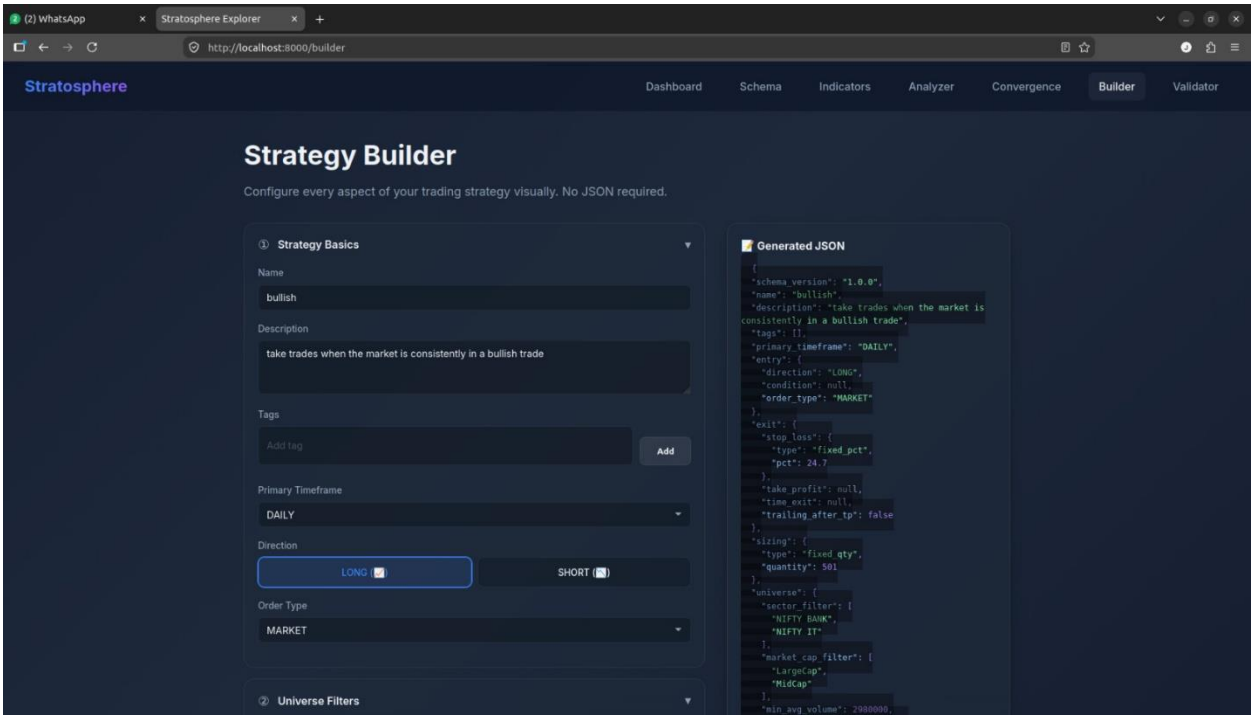


Fig 8.1 Strategy Builder Interface

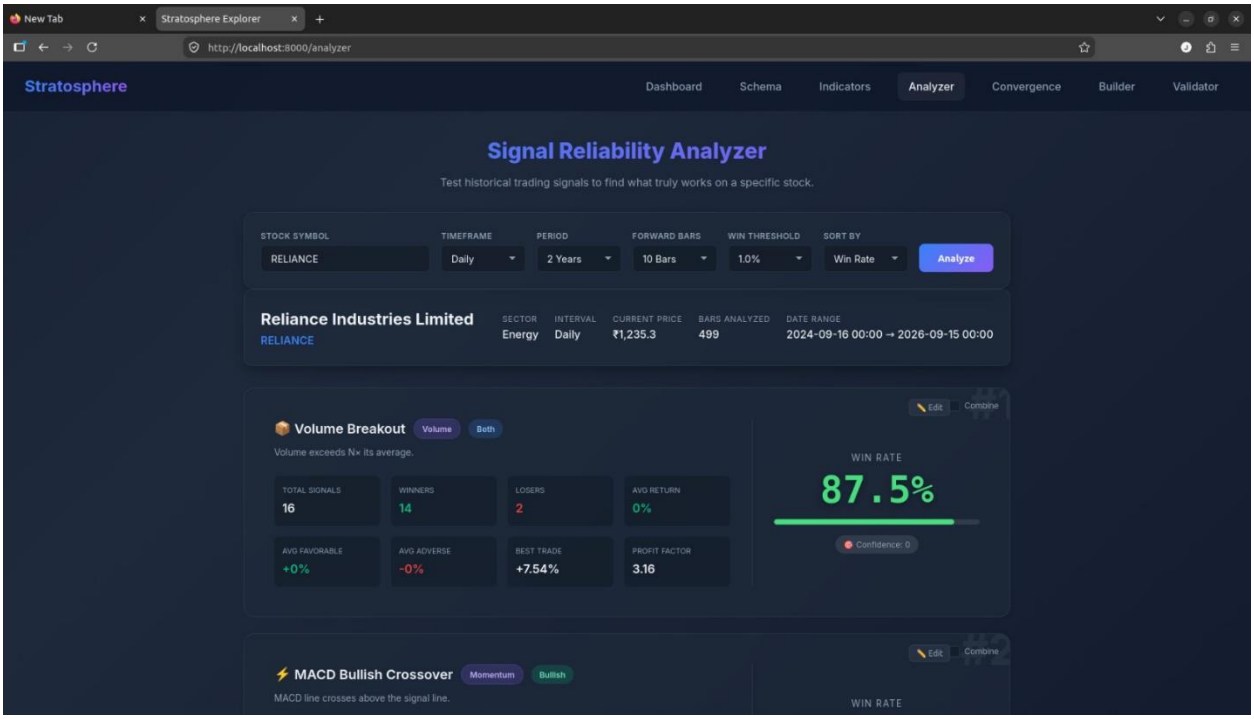


Fig 8.2 Signal Reliability Analyzer

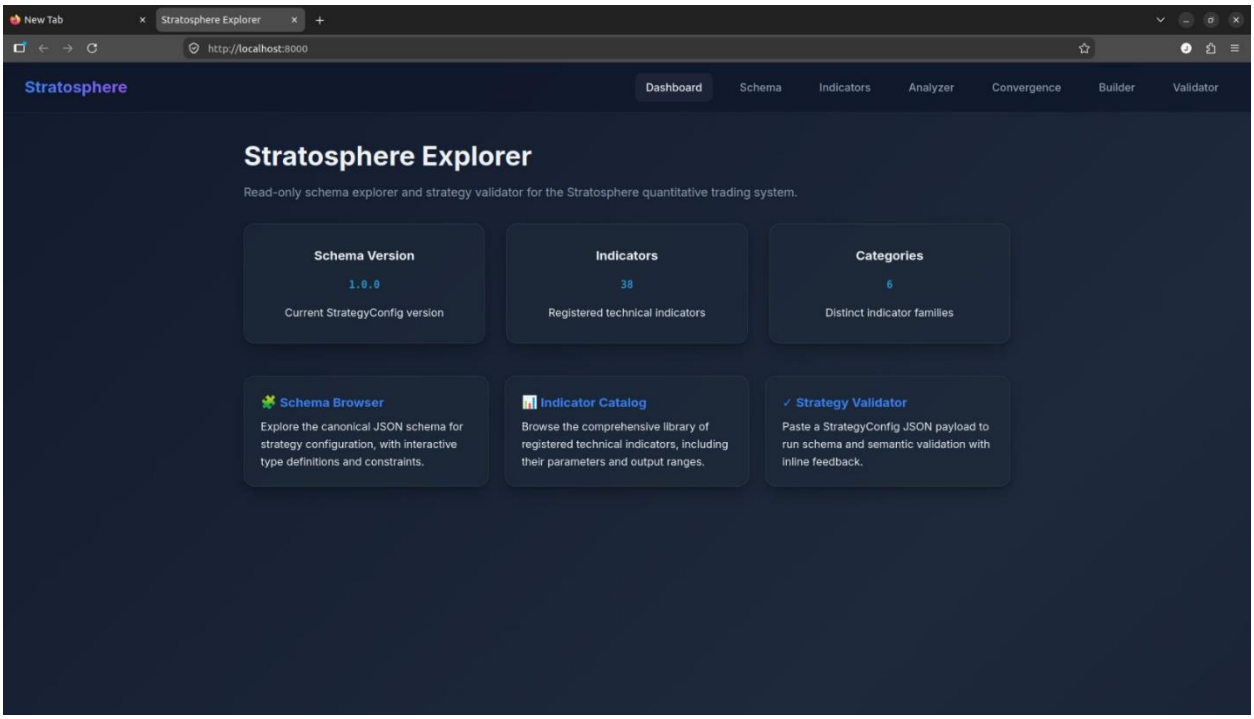


Fig 8.3 Stratosphere Explorer Dashboard

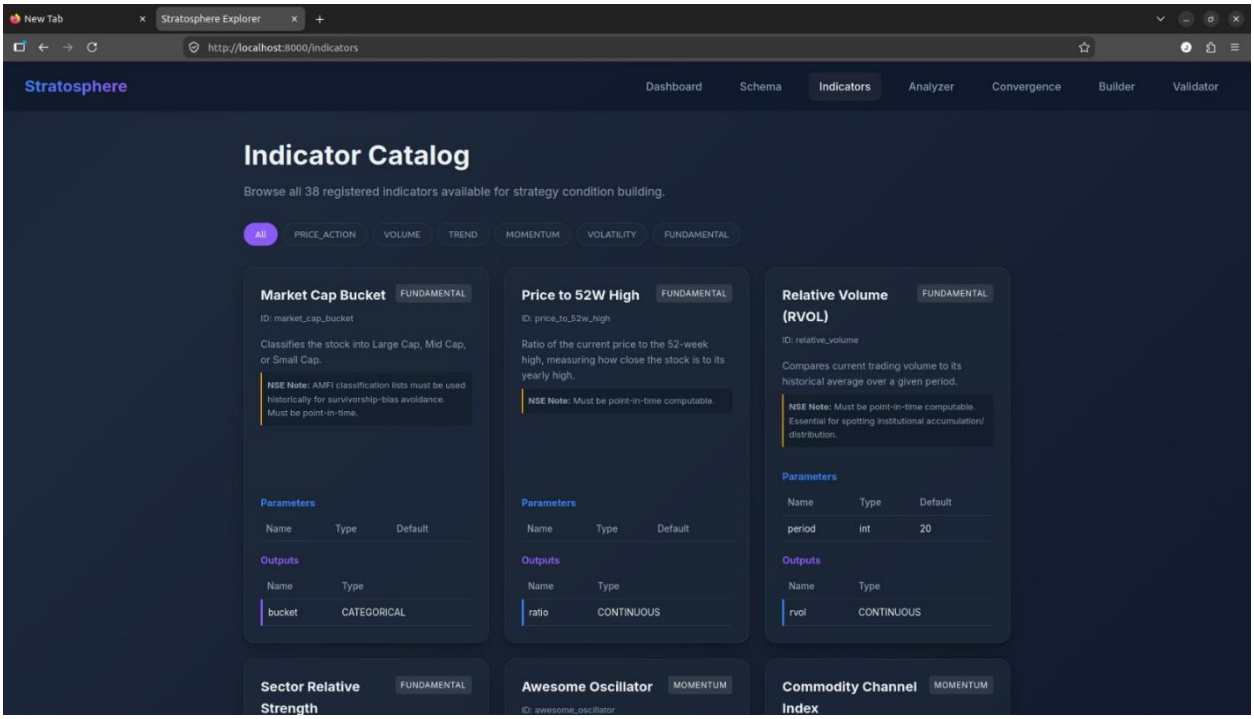


Fig 8.4 Indicator Catalog

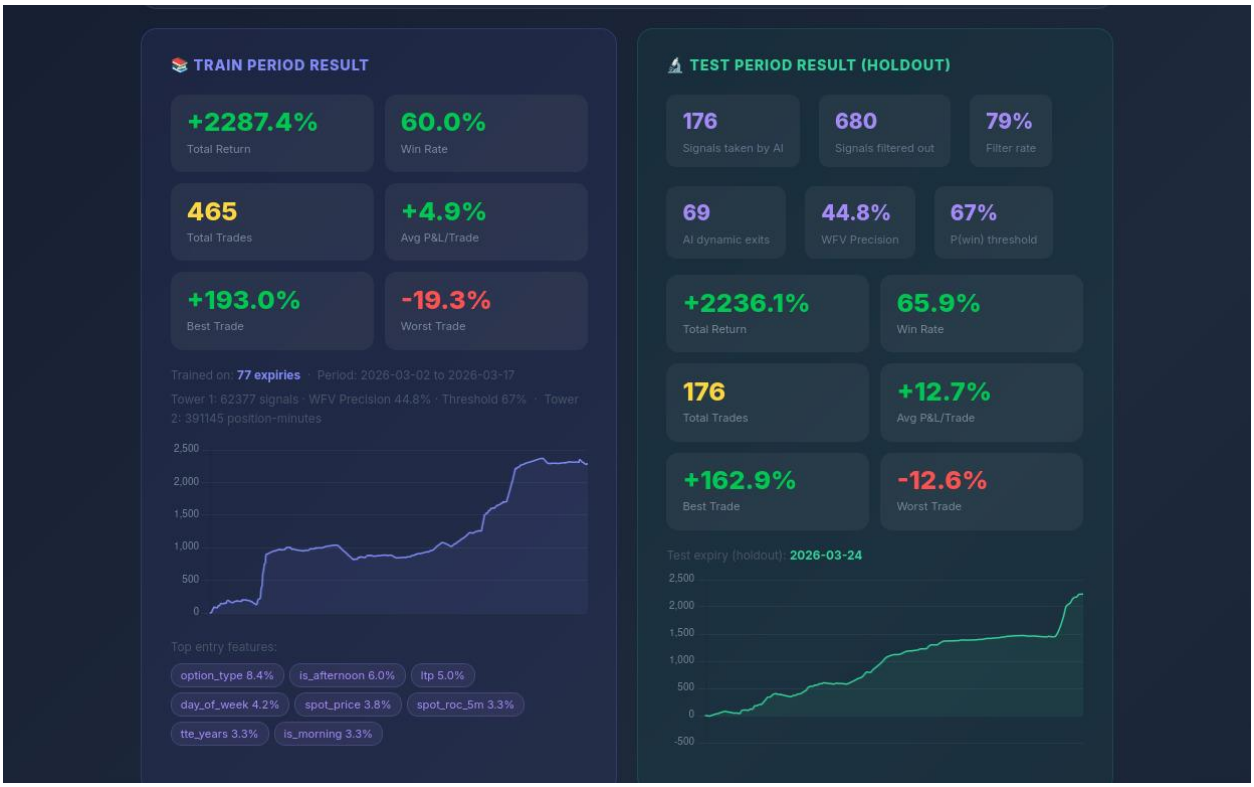


Fig 8.5 Train and Test Period Results

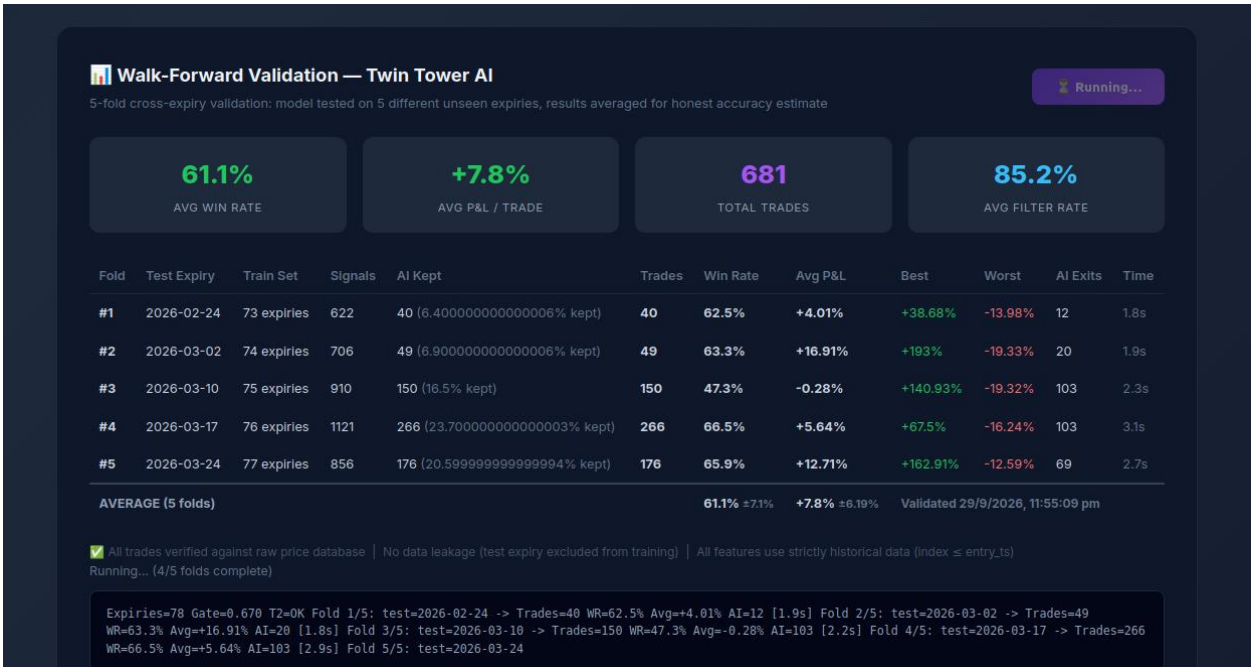
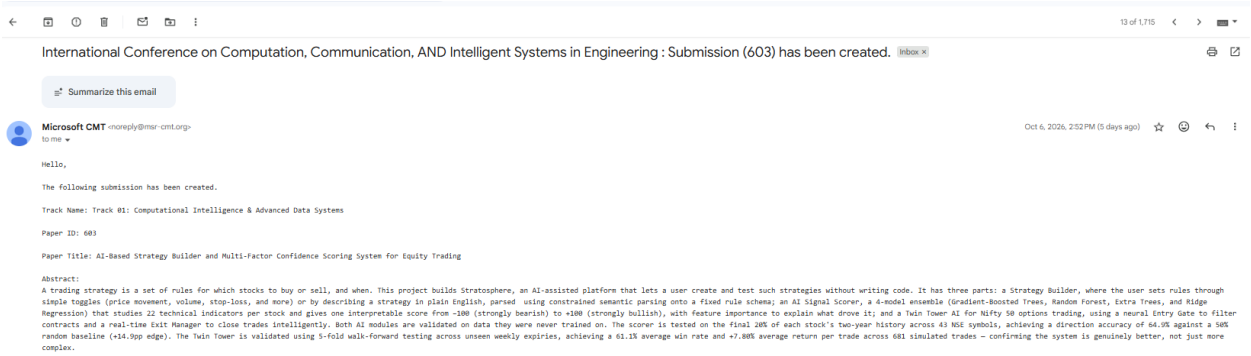


Fig 8.6 Walk-Forward Validation – Twin-Tower AI Results

CHAPTER 9

9. CONFERENCE



International Conference on Computation, Communication, AND Intelligent Systems in Engineering : Submission (603) has been created.

Summarize this email

Microsoft CMT <noreply@msr-cmt.org>
to me

Oct 6, 2026, 2:52 PM (5 days ago)

Hello,

The following submission has been created.

Track Name: Track 01: Computational Intelligence & Advanced Data Systems

Paper ID: 603

Paper Title: AI-Based Strategy Builder and Multi-Factor Confidence Scoring System for Equity Trading

Abstract:
A trading strategy is a set of rules for which stocks to buy or sell, and when. This project builds Stratosphere, an AI-assisted platform that lets a user create and test such strategies without writing code. It has three parts: a Strategy Builder, where the user sets rules through simple toggles (price movement, volume, stop-loss, and more) or by describing a strategy in plain English, parsed using constrained semantic parsing onto a fixed rule schema; an AI Signal Scorer, a 4-model ensemble (Gradient-Boosted Trees, Random Forest, Extra Trees, and Ridge Regression) that studies 22 technical indicators per stock and gives one interpretable score from -100 (strongly bearish) to +100 (strongly bullish), with feature importance to explain what drove it; and a Twin Tower AI for Nifty 50 options trading, using a neural Entry Gate to filter contracts and a real-time Exit Manager to close trades intelligently. Both AI modules are validated on data they were never trained on. The scorer is tested on the final 20% of each stock's two-year history across 43 NSE symbols, achieving a direction accuracy of 64.9% against a 50% random baseline (+14.9pp edge). The Twin Tower is validated using 5-fold walk-forward testing across unseen weekly expiries, achieving a 61.1% average win rate and +7.80% average return per trade across 681 simulated trades — confirming the system is genuinely better, not just more complex.

Fig 9.1 Conference Submission Confirmation Email

Created on: Tue, 06 Oct 2026 09:21:55 GMT

Last Modified: Tue, 06 Oct 2026 09:21:55 GMT

Authors:
- rrokith374@gmail.com (Primary)
- mani.se@vistas.ac.in
- jeyrenthshiva@gmail.com
- yeshwandh.d@gmail.com

Primary Subject Area: AI, Machine Learning & Deep Learning

Secondary Subject Areas: Not Entered

Submission Files:
ieee_paper.docx (333 Kb, Tue, 06 Oct 2026 09:14:43 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Fig 9.2 Conference Submission Details